Como funciona a configuração

Antes de começar, verifique se você atende a estes requisitos:

Suporte à linguagem

O Kit de Desenvolvimento de Agente (ADK) é compatível com várias linguagens de programação:

- **Python 3.11+** (abordado neste curso)
- **Java 17+** (consulte o Guia de início rápido do Java) (em inglês)

A linguagem dos exemplos neste curso é Python. Os conceitos se aplicam a todas as linguagens compatíveis.

Software necessário (Python)

- **Python 3.11 ou versão mais recente**: o ADK requer o Python 3.11 ou uma versão mais recente
    - Verifique a versão: `python --version` ou `python3 --version`
    - Se necessário, baixe a versão do site python.org.
- **pip**: instalador de pacotes Python (incluído no Python)
    - Verifique: `pip --version` ou `pip3 --version`
- **Terminal/prompt de comando**: acesso a uma interface de linha de comando
    - Mac: app Terminal
    - Windows: PowerShell ou prompt de comando
    - Linux: o terminal de sua preferência

Compatibilidade do sistema

- ✅ macOS
- ✅ Windows 10/11
- ✅ Linux (Ubuntu, Debian, Fedora etc.)

Considerações

Para este guia, consideramos que você:

- tem acesso de administrador/sudo para instalar pacotes;
- já conhece os comandos básicos de terminal;
- pode criar e editar arquivos de texto;
- tem conexão de internet para downloads de pacotes.

Observação: todos os comandos neste guia incluem variantes para outros sistemas operacionais quando eles são diferentes.

Configuração detalhada

Siga estas etapas para configurar o ambiente de desenvolvimento do ADK.

Etapa 1: verificar a instalação do Python

Primeiro verifique se você tem instalado o Python 3.11 ou uma versão mais recente.

Verifique a versão do Python:

```
python --version
```

ou

```
python3 --version
```

Resultado esperado: `Python 3.11.x` ou versão mais recente (por exemplo, `Python 3.12.0`)

Se o Python não estiver instalado ou a versão for muito antiga:

- Baixe a versão do site python.org.
- Instale o Python 3.11 ou uma versão mais recente.
- Reinicie o terminal após a instalação.
- Execute novamente a verificação de versão.

Etapa 2: criar um diretório de espaço de trabalho

Crie uma pasta dedicada para os projetos do ADK:

```
mkdir adk-workspace
cd adk-workspace
```

O que esse comando faz:

- Cria um novo diretório chamado `adk-workspace`.
- Alterna para esse diretório.
- Mantém todos os projetos de agente organizados em um único lugar.

Usuários do Windows: os comandos funcionam no PowerShell e no prompt de comando.

Etapa 3: criar um ambiente virtual

Um ambiente virtual isola as dependências do projeto da instalação do Python no sistema, evitando conflitos entre projetos diferentes.

Crie o ambiente virtual:

```
python -m venv .venv
```

Se o `python` não funcionar, tente:

```
python3 -m venv .venv
```

O que esse comando faz:

- Cria uma pasta chamada `.venv` com um ambiente Python isolado.
- Esse ambiente tem o próprio interpretador Python e espaço para o pacote.

Ative o ambiente virtual:

No Mac/Linux:

```
source .venv/bin/activate
```

No Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

No Windows (prompt de comando):

```
.venv\Scripts\activate.bat
```

Como saber se o ambiente está ativado:

- O prompt do terminal deve mostrar `(.venv)` logo no início.
- Exemplo: `(.venv) user@machine:~/adk-workspace$`

Importante: você precisa ativar o ambiente virtual sempre que abrir um novo terminal para trabalhar nos projetos de agente.

Etapa 4: instalar o ADK

Com o ambiente virtual ativado, instale o ADK:

```
pip install google-adk
```

O que é instalado:

- Framework do ADK com abstrações de agente (`LlmAgent`, `Runner` etc.)
- Ferramentas da CLI (comando `adk`)
- Dependências para trabalhar com modelos do Gemini do Google

Verifique a instalação:

```
adk --version
```

Resultado esperado: número da versão, como `1.0.0`, ou mais recente

Em caso de falha no comando `adk --version`:

- Verifique se o ambiente virtual está ativado (`(.venv)` deve aparecer no prompt).
- Tente desativar e reativar o ambiente: `deactivate` e depois o reative.
- Reinstale o ambiente: `pip install --force-reinstall google-adk`

Etapa 5: receber a chave de API

O agente precisa de acesso a um modelo de linguagem grande ou LLM (o componente "Modelo" do curso 1). Neste curso, vamos usar os modelos do Gemini do Google.

Há duas opções:

Opção A: API Gemini pelo Google AI Studio (recomendado para aprendizado)

Essa é a opção mais simples para começar.

Etapas:

1. Acesse o Google AI Studio.
2. Faça login com sua Conta do Google.
3. Clique em **Criar chave de API**.
4. Copie a chave de API (ela tem o formato `AIzaSyC...`).

Benefícios:

- ✅ Configuração rápida (basta a chave de API)
- ✅ Nível sem custo financeiro disponível
- ✅ Perfeito para desenvolvimento e aprendizado

Opção B: Gemini pela Vertex AI do Google Cloud (para produção)

Use essa opção se precisar de recursos empresariais ou se a implantação for em ambiente de produção.

Pré-requisitos:

- Conta do Google Cloud
- Faturamento ativado (teste sem custo financeiro disponível)
- Projeto do Google Cloud

Etapas:

1. Crie um projeto do Google Cloud, se ainda não tiver um.
2. Ative a API Vertex AI.
3. Instale a CLI gcloud.
4. Faça a autenticação.

```
gcloud auth application-default login
```

5. Anote o **ID do projeto** (visível no console do Cloud).
6. Anote o **Local** (por exemplo, `us-central1`).

Benefícios:

- ✅ Recursos de nível empresarial
- ✅ Melhor dimensionamento e cotas
- ✅ Segurança pronta para produção

Para simplificar, recomendamos a opção A (Google AI Studio) neste curso.

Etapa 6: criar o primeiro projeto de agente

O ADK oferece um comando para organizar um novo projeto de agente com a estrutura correta.

Crie o projeto:

```
adk create my_first_agent
```

O que esse comando cria:

```
my_first_agent/
├── agent.py      # Código principal do agente (você edita esta parte)
├── __init__.py   # Inicialização do pacote Python
└── .env          # Variáveis de ambiente (depois você edita esta parte)
```

Conceitos básicos sobre os arquivos:

- **`agent.py`**: é nele que você define o agente usando um código Python. Esse arquivo

reúne os componentes de modelo, ferramentas e orquestração que você estudou no curso 1. Ele contém a definição de `root_agent`.

- **`__init__.py`**: arquivo de inicialização do pacote Python que importa o módulo do agente. Ele é necessário para o ADK descobrir o agente.

- **`.env`**: um arquivo especial para armazenar informações sensíveis, como chaves de API. O ADK carrega esse arquivo automaticamente, mantendo os secrets fora do código.

Navegue até o projeto:

```
cd my_first_agent
```

Verifique o código do agente padrão:

Abra o `agent.py` no editor de texto. Vai aparecer algo semelhante a isto:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='Um agente assistente colaborativo.',
    instruction='Você é um assistente colaborativo.'
)
```

O que esse código faz:

- Importa a classe `Agent` do ADK.
- Cria um `root_agent` (o agente principal que o ADK procura).
- Faz a configuração com:
  - **Modelo**: `gemini-2.5-flash` (o LLM que permite o raciocínio)
  - **Nome**: `root_agent` (identificador obrigatório)
  - **Descrição**: o que o agente faz (usado em sistemas multiagente)
  - **Instrução**: como o agente deve se comportar

Este é um agente funcional mínimo. No módulo 2, você vai aprender a personalizar esses parâmetros e adicionar ferramentas.

Observação: é usado Python neste curso. Se você trabalha com Java, consulte o Guia de início rápido do Java (em inglês) para conferir a estrutura e a configuração específicas do projeto nessa linguagem.

Etapa 7: configurar a chave de API

Agora configure o arquivo `.env` com a chave de API que você recebeu na etapa 5.

Abra o arquivo `.env` no editor de texto.

Se você usa o Google AI Studio (opção A):

Substitua o conteúdo por:

```
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=your-actual-api-key-here
```

Substitua `your-actual-api-key-here` pela chave de API do Google AI Studio.

Exemplo:

```
GOOGLE_GENAI_USE_VERTEXAI=0
GOOGLE_API_KEY=AIzaSyC4lW...
```

Se você usa a Vertex AI (opção B):

Substitua o conteúdo por:

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=us-central1
```

Substitua:

- `your-project-id` pelo ID do projeto do Google Cloud;
- `us-central1` pelo local de sua preferência (se for diferente).

Exemplo:

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=my-agent-project-123
GOOGLE_CLOUD_LOCATION=us-central1
```

Observação importante sobre segurança:

- ⚠️ Nunca faça commit do arquivo `.env` no controle de versões (Git).
- O arquivo `.env` deve estar listado em `.gitignore`.
- As chaves de API são credenciais sensíveis. Tenha o mesmo cuidado que você tem com as senhas.

Etapa 8: verificar a configuração

Vamos executar a interface da web do ADK para confirmar se está tudo funcionando.

Inicie a interface da web do ADK:

```
adk web
```

Resultado esperado:

```
INFO:     Started server process
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

O que esse comando faz:

- Inicia um servidor da web local em `http://localhost:8000`.
- Abre uma guia do navegador automaticamente.
- Apresenta uma interface visual para interagir com o agente.

Indicadores de sucesso: ✅ Nenhuma mensagem de erro no terminal. ✅ O navegador abre a interface do ADK. ✅ A interface da web mostra o nome do agente.

Se aparecerem erros:

❌ **"ModuleNotFoundError: No module named 'google.adk'"**

- O ambiente virtual não foi ativado.
- Correção: reative-o com o comando `source .venv/bin/activate` (Mac/Linux) ou `.venv\Scripts\Activate.ps1` (Windows)

❌ **"Invalid API key" ou erros de autenticação**

- Verifique se o arquivo `.env` tem os nomes corretos das variáveis.
- Para o Google AI Studio: use `GOOGLE_API_KEY` (NÃO `GEMINI_API_KEY`).
- Não deixe espaços extras ao redor do sinal `=`.
- Verifique se a chave de API está correta (copie do Google AI Studio e cole novamente).

❌ **"Command 'adk' not found"**

- O ambiente virtual não foi ativado ou o ADK não foi instalado.
- Correção: ative o ambiente virtual e execute o comando `pip install google-adk`.

Interrompa o servidor da web: pressione `Ctrl+C` no terminal para interromper o servidor quando você concluir o teste.

Conceitos básicos sobre a configuração

Vamos associar o que você acabou de criar aos conceitos do curso 1.

Três componentes do ambiente

Lembrete: **agente = modelo + ferramentas + orquestração**

1. Modelo (configurado na etapa 7)

- O arquivo `.env` configura o acesso ao Gemini.
- O Gemini é o LLM que permite o raciocínio e a tomada de decisões.
- Ele é o "cérebro" que entende a linguagem e faz escolhas.

2. Ferramentas (mais adiante no módulo 3)

- Funções que o agente pode chamar para realizar ações.
- Exemplos: pesquisar na web, ler arquivos e enviar e-mails
- Elas fazem a ponte entre o "saber" e o "fazer".

3. Orquestração (fornecida pelo ADK)

- O framework que executa o ciclo do agente.
- Gerencia: percepção → raciocínio → ação → verificação → repetição
- Você o instalou na etapa 4 (`pip install google-adk`).

Como os arquivos funcionam juntos

`agent.py`: tudo se conecta aqui:

Você estudou o código do agente padrão na etapa 6. Vamos associá-lo aos conceitos do curso 1:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',        # Modelo: o mecanismo de raciocínio (curso 1)
    name='root_agent',               # Identidade: identificador obrigatório
    description='Um agente colaborativo.',  # Finalidade: o que o agente faz
    instruction='Você é colaborativo.'   # Comportamento: como agir
    # Ferramentas: você vai adicioná-las no módulo 3
    # Orquestração: a classe Agent faz esse processo automaticamente
)
```

Detalhamento:

- **Modelo** (`gemini-2.5-flash`): o LLM que permite o raciocínio e a tomada de decisões
- **Ferramentas** (ainda não apresentadas): funções que o agente chama para realizar ações (módulo 3)
- **Orquestração**: a classe Agent executa automaticamente o ciclo Percepção → Raciocínio → Ação → Verificação

`.env`: credenciais seguras:

- Mantém as chaves de API fora do código
- O ADK o carrega automaticamente
- Nunca passa por commit no Git

`__init__.py`: inicialização do pacote:

- Transforma a pasta em pacote Python
- Importa o módulo do agente
- Necessário para o ADK descobrir agentes

Observação: o ADK também dá suporte à configuração com base em YAML (consulte Agent Config, em inglês) e Java (consulte Guia de início rápido do Java, em inglês). Este curso aborda os agentes com base em código Python.

Fluxo de trabalho de desenvolvimento

1. **Editar** o `agent.py`: definir o comportamento do agente
2. **Executar** o comando `adk web`: fazer o teste na interface da web
3. **Iterar**: fazer mudanças, atualizar e testar novamente

No futuro, você vai aprender outras maneiras de executar os agentes:

- `adk run`: interação baseada em terminal
- `adk api_server`: implantação como serviço de API